

Task 9: Implement Exceptions and Exceptional handling in python.

Date: 24/9/25

Aim: To implement [Exceptions] and Exceptional handling in python.

Problem 9.1: You are developing a python that process a list of students' grades. The program is designed to allow the user to select a grade by specifying an index number. However, you need to ensure that the program handles cases where the user inputs an index that is out of range, i.e, an index that does not exist in the list.

Algorithm:

1. Start the program
2. Initializes a list of grades e.g, [85, 90, 78, 92, 88].
3. prompts the user to enter the index of the grade they wish to view.
4. Attempts to display the grade at the specified index.
5. If the index is out of range, catches the `IndexError` and prints an error message, "Invalid index. Please enter a valid index!"

Program:

```
# initialize the list of grades
grades = [85, 90, 78, 92, 88]

# Display the grade list
print("Grades list:", grades)

# prompt the user to enter the index of the grade they want to view try:

index = int(input("Enter the index of the grade you want to view: "))

# Attempt to display the grade at the specified index
print(f"The grade at index {index} is: {grades[index]}")
except IndexError:
    # Handle the case where the index is out of range
    print("Invalid index. please enter a valid index.")
```


except value Error:

Handle the case where the input is not an integer.

print ["Invalid input. please enter a numerical index."]]

Problem 9.2. You are developing a Python. Calculator program that performs basic arithmetic operations. one of the key functionalities is to divide two numbers entered by the user. However, dividing by zero is not allowed and would cause the program to crash if not handled properly.

Algorithm:

1. Start the program.
2. prompt the user to enter two number : a numerator and a denominator.
3. Attempts to divide the numerator by the denominator.
4. If the denominator is zero, catches the zero Division Error and displays an error message: "Error: Division by zero is not allowed."

Program:

function to perform division.

def divide-numbers():

try:

prompt the user to enter the numerator

numerator = float(input("Enter the numerator:"))

prompt the user to enter the denominator

denominator = float(input("Enter the denominator:"))

Attempt to perform division

result = numerator / denominator

print (f"Result: {result}")

except Zero Division Error:

Handle division by zero error.

Output:

Grades List: [85, 90, 78, 92, 88]

Enter the index of the grade you want to view:

[88, 85, 82, 89, 87] Enter the index of the grade you want to view:

out put:

Enter the numerator: 10

Enter the denominator: 0

ERROR!

Error: Division by zero is not allowed.


```
print("Error: Division by zero is not allowed.")  
except ValueError:
```

```
# Handle invalid input that is not a number
```

```
print("Error: please enter valid numbers.")
```

```
# call the function to execute the division operation  
divide_numbers()
```

Problem 9.3:

You are building a python application to determine if a person is eligible to vote based on their age. According to the rules, only individuals who are 18 years or older are allowed to vote. To enforce this rule, you decide to create a custom exception called Invalid Age Exception, which will be raised whenever an age below 18 is entered.

Algorithm:

1. Define the custom exception
2. Prompt the user for input
3. Check if the age is below 18.
4. Raise an exception if the condition is met
5. Handle the exception with a custom error message

Program:

```
# define python user-defined exception
```

```
class InvalidAgeException(Exception):
```

```
    "Raised when the input value is less than 18"  
    pass
```

```
# you need to guess this number.
```

```
number = 12
```


Output: `print("Error: Division by zero is not allowed.")`

Enter a number: 15

exception occurred: Invalid Age

`print("Error: please enter valid number.")`

Call the function to execute the division operation
`divide_numbers()`

Problem 4.3:

You are building a Python application to determine if a person is eligible to vote based on their age. According to the rules, only individuals who are 18 years or older are allowed to vote. To enforce this rule, you decide to create a custom exception called `InvalidAgeException`, which will be raised whenever an age below 18 is entered.

Algorithm:

1. Define the custom exception
2. Prompt the user for input
3. Check if the age is below 18
4. Raise an exception if the condition is met
5. Handle the exception with a custom error message

Program:

```
# Define Python user-defined exception  
class InvalidAgeException(Exception):  
    "Raised when the input value is less than 18"  
  
if you need to guess the number:  
number = 18
```



```

input_num = int(input("Enter a number:"))
if input_num < number:
    raise InvalidAgeException
else:
    print("Eligible to Vote")

except InvalidAgeException:
    print("Exception occurred: InvalidAge")

```

VEL TECH	
EX No.	9
PERFORMANCE (5)	5
RESULT AND ANALYSIS (5)	5
VIVA VOCE (5)	5
RECORD (5)	5
TOTAL (20)	20
SIGN WITH DATE	

Result: Thus the program for implement exceptions and Exceptional handling is executed and verified successfully.